Лабораторная работа №3/1 Классы, конструктор и методы

Java — строго типизированный объектно-ориентированный язык программирования, поэтому такие понятия как “Класс” и “Объект” играют ключевую роль в этом языке.

Класс - это шаблон, описание объекта, который содержит свойства (переменные), конструкторы, методы и внутренние классы. Объект - это экземпляр класса, создается через ключевое слово **new**.

1. Создание класса

Класс создается с помощью ключевого слова **class.** В теле класса можно определить переменные. Создадим, например, класс Word (Слово), он будет иметь две переменные: массив букв и язык, на котором оно написано.

```
3   public class Word {
4       char[] letters; // массив букв
5       String lang; // язык (RU, EN, ES ...)
6   }
7
```

Классы определяются в разных файлах, но можно и создать классы в одном файле тогда модификатор **public** может иметь только один класс.

Создадим метод **print()**, который будет выводить информацию о полях класса в консоль. Метод *toString(T[ ] t)* у **Arrays** конвертирует массив в строку.

```
3   import java.util.Arrays;
4
5   public class Word {
6       char[] letters; // массив букв
7       String lang; // язык (RU, EN, ES ...)
8
9       public void print() {
10          System.out.println("Word: " + Arrays.toString(letters)
11                  + " Lang: " + lang);
12      }
13  }
```

Попробуем создать экземпляр класса Word и вызвать метод print()

```java
public class Main {

    public static void main(String[] args) {
        Word word = new Word();
        word.print();
    }
```

Также можно вызвать метод без объявления переменной

```java
public class Main {

    public static void main(String[] args) {
        new Word().print();
    }

```

Но чаще всего используется первый вариант, т.к. приходится вызывать не один метод, а несколько. Ключевое слово **new** создает новый экземпляр класса.

Запустим программу и увидим, что вместо ожидаемого результата (массива букв

```
Word: null Lang: null
```

и языка) в консоль вывелось **null.**

Null - это специальный литерал, который обозначает, что этого типа не существует. Он применим к ссылочным типам данных, это могут быть классы, массивы, интерфейсы. Т.е. ссылочные типы данных ссылаются на объект в памяти и если они этого не делают, например, при объявлении переменной, то они имеют значение **null.**

Исправим это, присвоив значения к переменным класса.

```java
import java.util.Arrays;

public class Word {
    char[] letters = new char[] {'l', 'e', 't', 't', 'e', 'r'}; // массив букв
    String lang = "EN"; // язык (RU, EN, ES ...)

    public void print() {
        System.out.println("Word: " + Arrays.toString(letters)
                + " Lang: " + lang);
    }
}
```

```
Word: [l, e, t, t, e, r] Lang: EN
```

Но, что если нам нужно несколько таких классов с разными словами? Чтобы не создавать новый класс под каждое слово нам нужно заполнять данными вне этого

класса, например, в другом классе. Для этого уберем инициализацию переменных в классе Word и заполним их в классе Main, где мы создали его экземпляр.

```
4  public class Main {
5
6      public static void main(String[] args) {
7          Word word = new Word();
8          word.lang = "EN";
9          word.letters = new char[] {'l', 'e', 't', 't', 'e', 'r'};
10         word.print();
11     }
```

```
Word: [l, e, t, t, e, r] Lang: EN
```

Также можно создать еще один класс, но с другими значениями

```
4  public class Main {
5
6      public static void main(String[] args) {
7          Word word = new Word();
8          word.lang = "EN";
9          word.letters = new char[] {'l', 'e', 't', 't', 'e', 'r'};
10         word.print();
11
12         Word wordRU = new Word();
13         wordRU.lang = "RU";
14         wordRU.letters = new char[] {'с', 'л', 'о', 'в', 'о'};
15         wordRU.print();
16     }
```

```
Word: [l, e, t, t, e, r] Lang: EN
Word: [с, л, о, в, о] Lang: RU
```

Таким образом мы решили проблему повторного создания класса, использовав один класс как шаблон для создания новых объектов. Но существует другая проблема, что если нам нужно чтобы значения были заданы один раз и они оставались неизменными до конца работы программы. В подходе выше мы можем изменить данные, например язык.

```java
public class Main {

    public static void main(String[] args) {
        Word word = new Word();
        word.lang = "EN";
        word.letters = new char[] {'l', 'e', 't', 't', 'e', 'r'};
        word.lang = "RU";
        word.print();
    }
```

```
Word: [l, e, t, t, e, r] Lang: RU
```

2. Модификаоторы доступа

Для решения этой задачи существуют модификаторы доступа и конструктор. Модификаторы доступа - ограничивают область видимости сущности. В Java их четыре: public, default (package visible), protected, private**.**

1. **public** - к сущности можно обратиться из любого места программы.
2. **default (package visible)** - к сущности можно обратиться в рамках папки (package) где она находится, но вне она недоступна.
3. **protected** - как default, но еще доступ имеют классы наследники. Наследование рассмотрим в одной из следующих лабораторных.
4. **private** - к сущности можно обратиться в рамках класса.

Добавим модификатор доступа **private** к переменным класса Word.

```java
import java.util.Arrays;

public class Word {
    private char[] letters; // массив букв
    private String lang; // язык (RU, EN, ES ...)

    public void print() {
        System.out.println("Word: " + Arrays.toString(letters)
                + " Lang: " + lang);
    }
}
```

Теперь в классе Main IDE подчеркивает красным в местах, где идет обращение к переменным.

```java
public class Main {

    public static void main(String[] args) {
        Word word = new Word();
        word.lang = "EN";
        word.letters = new char[] {'l', 'e', 't', 't', 'e', 'r'};
        word.lang = "RU";
        word.print();
    }
```

3. Конструктор

Для решения проблемы выше создадим в классе Main конструктор. Это конструктор с аргументами.

```java
    public Word(char[] letters, String lang) {
        this.letters = letters;
        this.lang = lang;
    }
```

Конструктор – специальная конструкция, которая выполняет функцию инициализации объекта. Существуют пять типов конструкторов:

1. **по умолчанию** - не виден разработчику, но создается JVM, когда нету других конструкторов;

2. **без параметров** - создает сам разработчик, но не содержит аргументов.

```java
    public Word() {
        this.lang = "EN";
    }
```

3. **с аргументами** - содержит аргументы, используется для инициализации полей класса;

4. **конструктор копирования** - в качестве аргумента передается объект этого же класса;

```java
    public Word(Word word) {
        this.letters = word.letters;
        this.lang = word.lang;
    }
```

5. **связный конструктор** - в теле конструктора вызывается конструктор.

```
23      public Word(char[] letters) {
24          this(letters, lang: "EN");
25      }
```

6. С 17 версии Java появился встроенный **канонический конструктор** (конструктор со всеми аргументами класса)**,** он используется в **record** (подробнее рассмотрим ниже)**.**

4. this

Ключевое слово **this** - обозначает, что обращение к сущностям идет в рамках этого класса. Обычно его использую в конструкторе при инициализации поля (как в премерах выше), когда имя аргумента совпадает с именем поля. Или используют, чтобы создать связанный конструктор.

Вернемся к созданию объекта. Создадим объект и передадим аргументы в конструктор.

```
5
6       public static void main(String[] args) {
7           Word word = new Word(new char[] {'l', 'e', 't', 't', 'e', 'r'}, lang: "EN");
8           word.print();
9       }
```

```
Word: [l, e, t, t, e, r] Lang: EN
```

Вывод программы не изменился, но теперь мы достигли иммутабельности объекта, т.е. данные нельзя изменить.

5. Методы

Благодаря модификатору доступа **private** мы не можем получить доступ к объекту за пределами класса где он объявлен. Но, что если нам нужно получить значение переменной где-то в другом классе. Для этого мы можем создать метод, который будет возвращать значение переменной *letters*.

```
14      public char[] getLetters() {
15          return letters;
16      }
```

Метод - это часть кода, вынесенная в отдельный блок. Он может быть полезен в случае переиспользования, когда какой-либо блок кода используется в разных частях программы много раз. Метод может возвращать результат с помощью ключевого слова **return**, либо нет.

Теперь вызовем метод в классе *Main*

```
Word word = new Word(new char[] {'l', 'e', 't', 't', 'e', 'r'}, lang: "EN");
System.out.println(word.getLetters().length);
word.print();
```

```
6
Word: [l, e, t, t, e, r] Lang: EN
```

Так как метод возвращает ссылку на переменную, то мы можем работать с ней как и с обычной переменной, например, вывести в консоль длину массива.

Что делать если нам все же нужно изменить значение переменной? Для этого надо создать метод, который будет изменять значение переменной.

Для примера добавим в класс Word переменную типа boolean, если true, то первая буква будет в верхнем регистре.

```
5  public class Word {
6      private char[] letters; // массив букв
7      private String lang; // язык (RU, EN, ES ...)
8      private boolean isUpperCaseFirstSymbol; // с большой буквы слово?
```

Создадим метод *setUpperCaseFirstSymbol*, который в качестве параметра принимает значение true или false и присваивает его переменной класса Word, далее выполняется условие, если isUpperCaseFirstSymbol это true и массив не пустой, то первая буква пишется с большой буквы.

```
24     public void setUpperCaseFirstSymbol(boolean isUpperCaseFirstSymbol) {
25         this.isUpperCaseFirstSymbol = isUpperCaseFirstSymbol;
26         if (isUpperCaseFirstSymbol && letters != null && letters.length != 0) {
27             letters[0] = Character.toUpperCase(letters[0]);
28         }
29     }
```

Вызовем этот метод в классе Main и запустим программу.

```
6      public static void main(String[] args) {
7          Word word = new Word(new char[] {'l', 'e', 't', 't', 'e', 'r'}, lang: "EN");
8          word.setUpperCaseFirstSymbol(true);
9          System.out.println(word.getLetters().length);
10         word.print();
11     }
```

```
6
Word: [L, e, t, t, e, r] Lang: EN
```

В результате первая буква пишется в верхнем регистре.

Все что проделано выше называется - инкапсуляция. Инкапсуляция - это объединение и сокрытие переменных, методов в одном классе, так чтобы для работы с

ним наружу был доступен необходимый минимум, обычно это методы, которые называются геттеры и сеттеры от слов в начале методов - get, set.

6. Сущность record

record - это класс, который предназначен для хранения данных. Он похож на обычный класс, но у него есть несколько отличий: уже созданные методы - toString, equals и hashcode, геттеры/сеттеры на поля и канонический конструктор. Канонический конструктор, это то что мы прописываем после имени record`а в круглых скобках.

Например, вот record записи с двумя полями:

```
record Rectangle(double length, double width) { }
```

Это краткое объявление прямоугольника эквивалентно следующему классу:

```
public final class Rectangle {
   private final double length;
   private final double width;

   public Rectangle(double length, double width) {
      this.length = length;
      this.width = width;
   }

   double length() { return this.length; }
   double width()  { return this.width; }

   // Реализация equals() и hashCode()
   //(об этих методах поговорим позже), которые определяют
   // что два объекта записи равны, если они
   // имеют один и тот же тип и содержат одинаковые
   // значения полей.
   public boolean equals...
   public int hashCode...

   // Реализация метода toString(), возвращающая строковое
   // представление всех полей класса записи,
   // включая их имена.
   public String toString() {...}
}
```

7. Рекурсия

Когда мы выполняли задания по типу нахождения факториала или поиск наибольшого общего делителя (НОД), в них использоали циклы. Второй вариант – это использовать методы, которые вызывают в теле сами себя.

Например, следующий код рекурсивно можно представить так:

```
int a = 24;
int b = 54;

while (b != 0) {
    int temp = b;
    b = b % a;
    a = temp;
}
```

Обычный код

```
public static int nod(int a, int b) {
    if (b == 0) return a;
    return nod(b, b: a % b);
}
```

Рекурсивный код

Трасировка алгоритма для чисел a = 24 и b = 54 будет выглядеть так:
nod(24, 54)
= nod (54, 54 % 24) = nod (54, 6)
= nod (6, 6 % 54) = nod (6, 6)
= nod (6, 6 % 6) = nod (6, 0)
= 6

Еще один пример – сумма цифр числа.

```
int a = 241;
int sum = 0;

while (a != 0) {
    sum += a % 10;
    a /= 10;
}
```

Обычный код

```
public static int sum(int num) {  1 usage
    if (num < 10) return num;
    return num % 10 + sum( num: num / 10);
}
```

Рекурсивный код

Таким образом можно сокращать код. Но не во всех случаях рекурсия – лучший способ, например, в подсчете чисел Фибоначчи рекурсия наоборот будет хуже работать, долго подсчитывать, особенно для больших чисел (50, 100 и т.д.).

Да, в этой небольшой методичке невозможно рассмотреть все темы классов и методов (перегрузка методов, varargs, ключевое слово static и прочее), поэтому прикрепляю ссылки на доп. литературу.

Литература

1. Классы и объекты - https://metanit.com/java/tutorial/3.1.php
2. Знакомство с классами: написание собственных классов, конструкторы - https://javarush.com/groups/posts/1949-znakomstvo-s-klassami-napisanie-sobstvennihkh-klassov-konstruktorih
3. Инкапсуляция в Java - https://vertex-academy.com/tutorials/ru/inkapsulyaciya-java/
4. Record Classes - https://docs.oracle.com/en/java/javase/17/language/records.html

Задания

Задания выбирать: номер в списке группы mod 7

19 mod 7 = 5

За реализацию класса из C0 будет присвоено 9 баллов, за C1 – 12 баллов. За доп. задания со звездочкой будет прибавлено 2 балла

0. Recipe (Рецепт)

Атрибуты: название, автор, количество порций, ингредиенты (String[]), количество каждого ингредиента (double[]), шаги приготовления (String[]), время приготовления в минутах, вегетарианское ли блюдо.

Методы:
cook() — выводит шаги приготовления по порядку;
scaleServings(int newServings) — пересчитывает количество ингредиентов под новое число порций;
isLongRecipe() — возвращает true, если время приготовления больше 60 минут.

* В геттере ингредиентов вывести все ингредиенты с количеством, например:
1. Мука — 200.0 г

1. Smartphone (Смартфон)

Атрибуты: модель, производитель, уровень заряда (0–100), объём памяти в ГБ, объём ОЗУ в ГБ, включён ли, установленные приложения (String[]).

Методы:
turnOn() / turnOff();
charge(int minutes) — увеличивает заряд;
installApp(String app) — устанавливает приложение, если хватает памяти;
useApp(String app) — уменьшает заряд, если приложение установлено.

* В методе charge выводить уровень заряда каждую минуту, пока телефон не зарядится до 100 или не закончится время.

2. Student (Студент)
Атрибуты: ФИО, группа, возраст, средний балл, список долгов (String[]), получает ли стипендию.

Методы:
addDebt(String subject) — добавляет долг;
payDebt(String subject) — убирает долг;
canGetScholarship() — возвращает true, если долгов нет и средний балл ≥ 4.5;
getInfo() — выводит краткую информацию о студенте.

* В сеттере среднего балла проверять диапазон 2.0–5.0. Если значение некорректное — не менять поле.

3. CoffeeMachine (Кофемашина)
Атрибуты: бренд, количество воды в мл, количество кофе в граммах, количество молока в мл, включена ли, сколько чашек приготовлено.

Методы:
turnOn() / turnOff();
addWater(int ml), addCoffee(int g), addMilk(int ml);
makeEspresso() — проверяет воду и кофе, уменьшает запасы, увеличивает счётчик чашек;
makeLatte() — проверяет воду, кофе и молоко;
clean() — сбрасывает счётчик чашек.

* В makeLatte при нехватке ингредиентов выводить, чего именно не хватает, и не готовить кофе.

4. Movie (Фильм)
Атрибуты: название, режиссёр, год выпуска, длительность в минутах, жанр, рейтинг, просмотрен ли, отзывы (String[]).

Методы:
watch() — отмечает фильм как просмотренный;
addReview(String review) — добавляет отзыв;
getAverageReviewLength() — возвращает среднюю длину отзывов
isNewFilm(int currentYear) — возвращает true, если фильм выпущен не более 5 лет назад.

* В геттере отзывов вывести все отзывы по порядку с номерами.

5. Elevator (Лифт)

Атрибуты: текущий этаж, минимальный этаж, максимальный этаж, открыты ли двери, максимальный вес, текущий вес, количество пассажиров.

Методы:
openDoors() / closeDoors();
goToFloor(int floor) — перемещает лифт, если двери закрыты и нет перегруза;
load(int weight) — добавляет вес;
unload(int weight) — уменьшает вес.

* В goToFloor выводить каждый этаж, который лифт проезжает, например:
Этаж 3... Этаж 4... Этаж 5...

6. WeatherStation (Метеостанция)
Атрибуты: город, текущая температура в Цельсиях, влажность, давление, измерения температуры за последние N дней (double[]).

Методы:
toFahrenheit() — переводит текущую температуру в Фаренгейт;
toCelsius() — переводит текущую температуру в Цельсий;
addMeasurement(double temp) — добавляет новое измерение;
getAverageTemperature() — средняя температура за N дней;
getMaxTemperature() — максимальная температура за N дней.

* В геттере измерений вывести все значения температуры за последние N дней.

C0 Создайте класс MyList (список) с такими полями, как начальный размер элементов (по умолчанию = 10), массив элементов.
Реализуйте методы:
1. добавления элемента в список;
2. поиск элемента по индексу;
3. удаление всех элементов;
4. удаление конкретного элемента;
5. обновление элемента по индексу;
6. получение элемента по индексу;
7. добавление нового списка в существующий;
8. проверка на пустоту;
9. метод вывода всех элементов в консоль.
Предусмотрите приватный метод автоматического расширения внутреннего массива и удалению элемента по индексу. Список может хранить конкретный тип данных, а может и все сразу (Object). Решите задачу палиндрома из предыдущей лабораторной, используя этот класс.

* Создайте методы сортировки списка (учесть, что сортировка может быть не только чисел), обрезки (указать первую позицию и последнюю, это будет промежуток который

будет удален из списка) и метод, который определяет существует ли элемент в списке или нет.

```
Пример:
// создание списка
MyList list = new MyList();

// добавление элементов по одному
list.add(1);
list.add(2);

// добавление элементов всего списка
list.addAll(new MyList(1, 2, 3, 4, 5, 6, … ));

// удаление элемента по индексу
list.remove(0);

// вывод в консоль
list.toString();

//удаление всех элементов
list.removeAll();
```

С1. Создайте класс MyUniqueList (список уникальных элементов) с такими полями, как начальный размер элементов (по умолчанию = 10), массив элементов.
Реализуйте методы:
1. добавления элемента в список
2. добавление нового списка в существующий
3. удаление всех элементов
4. удаление конкретного элемента
5. обновление элемента по индексу
6. получение элемента по индексу
7. проверка на пустоту
8. метод вывода всех элементов в консоль.

Предусмотрите приватный метод автоматического расширения внутреннего массива, а также обеспечьте гарантию, что все элементы в списке будут уникальны. Список может хранить конкретный тип данных, а может и все сразу (Object) (включая ссылочные типы данных).
* Создайте методы сортировки списка (учесть, что сортировка может быть не только чисел), обрезки (указать первую позицию и последнюю, это будет промежуток который будет удален из списка) и метод, который определяет существует ли элемент в списке или нет.

```
Пример:
// создание списка
MyUniqueList list = new MyUniqueList();

// добавление элементов по одному
```

```
list.add(new Person("Vasya"));
list.add(2);

// добавление элементов всего списка
list.addAll(new MyUniqueList(1, 2, 3, 4, 5, 6, … ));

// удаление элемента по индексу
list.remove(0);

// получение элемента по индексу
list.get(4);

// вывод в консоль
list.toString();

//удаление всех элементов
list.removeAll();
```

Примерный список общих вопросов:

1. Что такое класс? Что такое объект? В чём разница между классом и объектом?
2. Что такое конструктор? Зачем он нужен? Какие виды конструкторов бывают?
3. Что такое конструктор по умолчанию? Когда компилятор создаёт его автоматически? Что произойдёт, если объявить свой конструктор?
4. Можно ли перегружать конструкторы? Как вызвать один конструктор из другого? Какие ограничения у this(...)?
5. Что такое сигнатура метода? Входит ли в неё возвращаемый тип? Влияют ли имена параметров?
6. Зачем нужны модификаторы доступа? В чём разница между public, private, default (package-private), protected?
7. Что такое инкапсуляция? Почему поля обычно делают private? Зачем нужны геттеры и сеттеры?
8. Ключевое слово this: где и зачем используется? Можно ли использовать this в конструкторе, методе, сеттере?
9. Перегрузка методов: что это? Как компилятор выбирает подходящий метод? Можно ли перегружать методы только по возвращаемому типу?
10. Varargs: как работает Object...? Можно ли использовать в конструкторах? Какие ограничения? (один varargs, только последний параметр)
11. Что такое static? Когда использовать static поля и методы? Можно ли из static метода обращаться к нестатическим полям?
12. Что такое final? Для чего final у полей, методов, локальных переменных? Что такое константы?
13. record: когда применяется? Какие методы генерируются автоматически? Можно ли добавлять свои методы? Можно ли использовать в заданиях?
14. Рекурсия: что это? Где может пригодиться в ваших заданиях? Какие риски? (StackOverflowError)
15. Как передать логику сортировки или сравнения без лямбд? (анонимные классы, Comparable, Comparator). В чём разница между Comparable и Comparator?

16. Обработка null: как сравнивать, искать, добавлять? Чем опасен NullPointerException? Как его избегать?
17. Оценка сложности операций: O(1), O(n), O(n^2). Как оценить сложность методов в вашем классе?
18. BigDecimal: зачем нужен? Чем отличается от double? Как округлять? (RoundingMode)
19. Массивы в Java: как создать, как получить длину, как пройти? Какие ограничения у массивов?
20. Строки: почему String неизменяемый? Чем отличается == (два равно) от equals? Что такое пул строк?
21. Что такое пакет? Зачем нужны пакеты? Как связаны пакеты и модификатор default?
22. Как работает передача параметров в Java? Передача по значению или по ссылке? Что передаётся для объектов?
23. Что такое immutable класс? Как сделать класс неизменяемым? Зачем это нужно?
24. Как протестировать класс без main? Что такое JUnit? (если проходили)